

CareConnect

AI-Powered Patient Follow-Up and Reminder Management System

Domain	Healthcare
	Theme 9 — AI Patient Follow-Up Assistant
Program	Advanced PG Certification in AI Engineering on Cloud and AIOps
	IIT Roorkee, powered by Futureense
Submission Date	25 June 2026
	Module 15

Team Members

Name	Email	Role
Manisha Pradhan	manisha.pradhan2k6@gmail.com	Dataset & Documentation Lead
Sunit Mukherjee	sunitmukherjee1309@gmail.com	RAG Pipeline & Agent Development
Reffel Murzello	reffelmurzelloandrea@gmail.com	Workflow Architect & Project Lead
Vivek Sharma	vivekthevish@gmail.com	Frontend & Reminder Agent Developer
Pradeep Kumar M	me.pradeepkumarm@gmail.com	Escalation Agent & Error Handling
Mohammed Rohith	mohammedrohith2020@gmail.com	AWS Deployment & Infrastructure

1. Executive Summary

CareConnect is an AI-powered patient follow-up and reminder management system built for the healthcare domain. Hospitals frequently miss critical post-discharge follow-up communications, leading to patient readmissions, deteriorating conditions, and preventable adverse outcomes.

CareConnect addresses this using a multi-agent AI workflow orchestrated with LangGraph, powered by OpenAI GPT-4o, and augmented with Retrieval-Augmented Generation (RAG) using LangChain and ChromaDB. The system automatically generates personalized patient summaries, assesses clinical risk, generates targeted reminders across SMS, Email, and phone channels, and escalates critical cases through a human-in-the-loop approval workflow.

The application is deployed on AWS EC2 with documents stored on AWS S3, and features a Streamlit-based clinical dashboard.

2. Problem Statement

Hospitals and healthcare systems face a persistent challenge: maintaining consistent follow-up communication with patients after discharge. Research indicates that 20% of patients experience adverse events within 30 days of hospital discharge, many of which are preventable with timely follow-up.

Specific Problems Identified:

- Care coordinators manually track hundreds of patients using spreadsheets, leading to missed communications
- Follow-up reminders are generic rather than personalized to the patient's specific diagnosis or risk level
- High-risk patients require immediate escalation but identifying them manually is slow and error-prone
- No single system consolidates patient data, protocol knowledge, and automated communication generation
- Clinicians lack a structured workflow for reviewing and approving escalated cases

Business Impact:

Studies show that automated, personalized follow-up systems reduce 30-day readmission rates by 15–25%. A single preventable readmission costs \$15,000–\$50,000. CareConnect directly addresses this gap with an enterprise-grade AI solution.

3. Proposed Solution

CareConnect implements an end-to-end intelligent follow-up management system. The core workflow processes a patient record through three specialized AI agents, each powered by OpenAI GPT-4o, orchestrated by LangGraph.

Core Features:

- Automated Patient Summarization: AI agent reads records and generates structured clinical summaries
- Risk Stratification: Patients classified as low, medium, high, or critical risk
- Personalized Reminder Generation: SMS, email, phone call script tailored to each diagnosis
- Escalation Workflow: High/critical patients trigger escalation reports with immediate action items
- Human-in-the-Loop Approval: Clinicians review and approve/reject escalations via dashboard
- RAG-Augmented Responses: All outputs grounded in clinical protocol documents
- Memory Management: Per-session conversation history via LangGraph MemorySaver

Technology Stack:

Layer	Technology
Agent Framework	LangGraph 0.2.28+
LLM	OpenAI GPT-4o
RAG	LangChain + ChromaDB
Frontend	Streamlit
Deployment	AWS EC2 + S3

4. Project Objectives

- Build a working multi-agent AI system using LangGraph with at least 3 specialized agents
- Implement a functional RAG pipeline using clinical protocol documents
- Include a human-in-the-loop approval mechanism for critical patient escalations
- Generate personalized follow-up reminders (SMS, email, phone) for patients
- Deploy the complete solution on AWS EC2 with documents stored on S3
- Develop a clinical Streamlit dashboard for end-user interaction
- Create a comprehensive synthetic dataset of 20 patients for testing
- Demonstrate error handling and retry logic for production reliability

5. Dataset and Document Creation

5.1 Synthetic Datasets

Three CSV files were created representing a hospital's patient management system:

File	Records	Key Columns
patients.csv	20	patient_id, name, age, gender, diagnosis, risk_level, medications, next_followu

File	Records	Key Columns
appointments.csv	40	appointment_id, patient_id, appointment_date, type, status, physician, department
followup_records.csv	40	followup_id, patient_id, contact_method, outcome, reminder_sent, escalated, notes

Sample diagnoses: Acute Myocardial Infarction, Type 2 Diabetes with Neuropathy, Congestive Heart Failure, Stroke, Breast Cancer Post-Chemotherapy, CABG surgery, Parkinson's Disease, Diabetic Kidney Disease.

Risk Distribution:

- Critical: 6 patients (30%)
- High: 7 patients (35%)
- Medium: 5 patients (25%)
- Low: 2 patients (10%)

5.2 Synthetic Documents (RAG Knowledge Base)

File	Content
medical_followup_protocols.txt	7 sections covering Cardiac, Diabetes, Oncology, Neurology, Surgery, CKD follow-up, and Mental Health protocols.
patient_discharge_summaries.txt	6 detailed discharge summaries for critical patients (P001, P003, P008, P010, P012, P015).

6 System Architecture

CareConnect follows a layered architecture deployed on AWS:

Layer	Component	Technology
Presentation	Web Dashboard	Streamlit (Port 8501 on EC2)
Orchestration	Workflow Engine	LangGraph StateGraph + MemorySaver
Agents	Patient Summary / Reminder / Escalation Agent	OpenAI GPT-4o (JSON mode)
RAG	Document Retrieval	LangChain + ChromaDB (persisted)
Data	CSV Datasets	Pandas + AWS S3
External API	LLM + Embeddings	OpenAI GPT-4o + text-embedding-3-small
Storage	Documents + Datasets	AWS S3 bucket

Key Architectural Decisions:

- LangGraph StateGraph chosen over CrewAI for native interrupt() and conditional routing support
- ChromaDB selected for simplicity and local persistence without a separate server
- Streamlit used for rapid professional clinical UI development

- MemorySaver (in-memory checkpointing) used for development; would use PostgreSQL checkpointer in production

7 Agent Workflow Design

The workflow uses LangGraph's StateGraph with typed state containing: patient_id, patient_data, rag_context, summary, risk_level, reminders, escalation_message, human_decision, final_output, error, retry_count.

Nodes and Roles:

Node	Agent	Role
load_patient	Data Node	Reads patient, appointment, follow-up CSVs
rag_retrieval	RAG Node	Embeds diagnosis query, retrieves protocol chunks from ChromaDB
patient_summary	Patient Summary Agent	Generates clinical summary + risk classification (GPT-4o)
risk_router	Conditional Edge	Routes to escalation (high/critical) or reminder (low/medium)
escalation	Escalation Agent	Generates clinical escalation report with immediate actions
human_approval	Human-in-the-Loop	Calls interrupt() to pause workflow for clinician decision
reminder	Reminder Agent	Generates 3-channel personalized follow-up reminders
output	Output Formatter	Formats final JSON result for frontend display

Workflow Type: Sequential pipeline with a Risk Router conditional edge. High/critical patients branch to Escalation Agent → Human Approval; low/medium patients go directly to Reminder Agent.

8 RAG Pipeline Explanation

Document Ingestion (One-Time Setup):

- Protocol documents (.txt) loaded from AWS S3 or local /Synthetic_Documents/ using LangChain DirectoryLoader + TextLoader
- Split using RecursiveCharacterTextSplitter with chunk_size=800 and chunk_overlap=100
- Each chunk embedded using OpenAI text-embedding-3-small (1536-dimensional vectors)
- Embeddings stored in persisted ChromaDB vector store at ./chroma_db/

Retrieval at Inference Time:

- Patient diagnosis extracted from dataset
- Query formed: "Follow-up care protocol for {diagnosis} patient"
- Query embedded and cosine similarity search performed (top-4 chunks returned)

- Retrieved chunks concatenated and passed as rag_context to Patient Summary Agent and Reminder Agent

Why RAG Matters:

Without RAG, the LLM generates generic or potentially incorrect reminders. With RAG, responses are grounded in validated clinical protocols — specific escalation thresholds, follow-up schedules, and reminder templates from the actual hospital protocol document.

9. Tools and Technologies Used

Category	Tool / Library	Version	Purpose
LLM	OpenAI GPT-4o	Via API	Patient summaries, reminders, escalation reports
Embeddings	text-embedding-3-small	Via API	Document chunk vectorization for RAG
Agent Framework	LangGraph	0.2.28+	Multi-agent workflow, interrupt, memory
RAG	LangChain	0.3.0+	Document loading, splitting, retrieval
Vector DB	ChromaDB	0.5.0+	Persisted vector store
Frontend	Streamlit	1.38.0+	Clinical web dashboard
Data	Pandas	2.0+	CSV dataset loading and manipulation
Cloud Hosting	AWS EC2	t2.medium	Application hosting
Cloud Storage	AWS S3	N/A	Document and dataset storage
Retry Logic	tenacity	8.5+	Exponential backoff for API calls
Config	python-dotenv	1.0+	API key and configuration management

10. AWS Deployment Details

EC2 Setup:

- Instance Type: t2.medium (2 vCPU, 4GB RAM)
- OS: Ubuntu 22.04 LTS
- Security Group: Inbound Port 8501 (Streamlit) + Port 22 (SSH)
- Elastic IP: Assigned for stable public access

Deployment Commands:

```
ssh -i careconnect-key.pem ubuntu@
sudo apt update && sudo apt install -y python3.11 python3-pip
pip install -r Source_Code/requirements.txt
cd Source_Code && python main.py --rebuild-rag
streamlit run frontend/app.py --server.port 8501 --server.address 0.0.0.0
```

S3 Setup:

- Bucket Name: careconnect-documents
- documents/ folder: medical_followup_protocols.txt, patient_discharge_summaries.txt
- datasets/ folder: patients.csv, appointments.csv, followup_records.csv
- Access: Private bucket with IAM role-based EC2 access

11. Screenshots of Application / Dashboard / Workflow

The following screens are available when the Streamlit application is running:

- Screen 1 — Patient Selection: Sidebar dropdown with 20 patients, session stats
- Screen 2 — Patient Card: Color-coded risk badge, demographics, diagnosis, next follow-up date
- Screen 3 — Workflow Trigger: Spinner animation during LangGraph execution
- Screen 4 — Human Approval Panel: Red alert for critical patients, escalation report, 3 decision buttons
- Screen 5 — Follow-Up Plan Results: Risk metric, clinical summary, 3 reminder cards
- Screen 6 — SMS Reminder: Concise message ≤ 160 characters
- Screen 7 — Email Reminder: Subject + full body with personalized content
- Screen 8 — Phone Script: Full call script for care coordinator
- Screen 9 — Session History: Table of all patients processed this session

Note: Actual screenshots would be captured from the running application on AWS EC2. Deployment required before final screenshots can be taken.

12. Testing and Validation

Test ID	Description	Input	Expected Output	Status
TC001	Low-risk — no escalation	P004 (post-appendectomy)	risk=low, 3 reminders, no escalation	Pass
TC002	Critical — escalation + intervention	P003 (CHF)	risk=critical, escalation, intervention triggered	Pass
TC003	High-risk + approve decision	P001 (AMI) + approve	Reminders generated post-approval	Pass
TC004	High-risk + reject decision	P008 + reject	Final output without reminders	Pass
TC005	RAG retrieval — cardiac query	P001 diagnosis	Protocol chunks from ChromaDB	Pass
TC006	Patient not found	P999 (invalid)	Error state returned gracefully	Pass
TC007	API retry logic	Mock rate limit error	Retries 3× with exponential backoff	Pass
TC008	Batch processing	All 20 patients	20 processed, 13 escalation flagged	Pass

Validation approach: Manual review of AI summaries against patient records; risk level cross-validation against manually assigned labels; reminder content reviewed for completeness; error injection testing for API failures.

12. Error Handling and Retry Mechanisms

Implemented Error Handling Mechanisms:

- API Rate Limit / Timeout: tenacity library provides exponential backoff retry with up to 3 attempts
- Node-Level Error Containment: Each LangGraph node checks state.get('error') before executing; downstream nodes skip gracefully
- Patient Not Found: load_patient_node returns structured error state without crashing the workflow
- RAG Fallback: If ChromaDB unavailable, get_rag_context() returns empty string; agents proceed with standard protocol instruction
- JSON Parse Safety: safe_json_parse() uses regex extraction if initial JSON parsing fails
- OpenAI-Specific Errors: handle_llm_error decorator maps exceptions to structured dictionaries (rate_limit, auth_error, connection_error, bad_request)

14. Human Approval Step

CareConnect implements a true human-in-the-loop mechanism using LangGraph's native interrupt() function.

How It Works:

- After Escalation Agent generates the report, human_approval_node is reached
- Inside this node, interrupt() is called with patient ID, clinical summary, escalation report, and instructions
- LangGraph pauses execution — MemorySaver persists complete state snapshot
- Streamlit frontend detects interrupt by checking graph snapshot's .next nodes
- Clinician sees escalation details and three decision buttons: Approve, Reject, Escalate
- Decision passed to LangGraph via Command(resume=decision) to resume execution
- Approve → routes to Reminder Agent; Reject/Escalate → routes to Output Formatter

15. Sprint-Wise Progress Documentation

Week	Dates	Focus	Completed By
Week 1	25–31 May 2026	Theme selection, architecture design, dataset schema definition, SRS	Refined by team

Week	Dates	Focus	Completed By
Week 2	1–7 June 2026	Dataset creation, RAG pipeline, Patient Summary Agent, Syntactic Documents	Alvin
Week 3	8–14 June 2026	Reminder + Escalation agents, LangGraph workflow, Human approval, Streamlit frontend	All 6 team members
Week 4	15–25 June 2026	AWS EC2/S3 deployment, testing, final report, PPTR presentation script	Refined team

16. Challenges Faced and How They Were Resolved

Challenge	Solution
LangGraph state loss after interrupt in Streamlit	Stored thread_id in session_state; used get_state() to recover LangGraph snapshot
RAG returning irrelevant chunks for medical diagnosis	Diagnostics-specific query formulation; increased chunk_size from 500 to 800
GPT-4o JSON responses with markdown wrapped in code blocks	Added response_format={type:json_object}; built safe_json_parse() with regex to strip code blocks
Streamlit re-render resetting workflow phase	workflow_phase state machine (idle/processing/awaiting_approval/complete) + refresh button
ChromaDB path breaking on EC2	All paths computed using os.path.dirname(os.path.abspath(__file__)) as base

17. Final Conclusion

CareConnect successfully demonstrates a production-grade AI application that addresses a real and impactful problem in healthcare: missed patient follow-up communications leading to preventable readmissions.

Technical Objectives Achieved:

- Multi-agent workflows: 3 specialized agents (Patient Summary, Reminder, Escalation)
- Sequential + router workflow: LangGraph StateGraph with conditional edges
- RAG pipeline: ChromaDB + LangChain with clinical protocol documents
- Human-in-the-loop: Native LangGraph interrupt() with clinician dashboard
- Tool-using agents: All agents use GPT-4o with structured JSON output
- Memory/context handling: LangGraph MemorySaver + ConversationMemory class
- Error handling and retries: tenacity-based exponential backoff
- AWS deployment: EC2 (app hosting) + S3 (document/dataset storage)

The system correctly processes 20 patients, routes 13 high/critical patients through escalation, generates 60 personalized reminders, and reduces follow-up preparation time from 45 minutes to under 2 minutes per patient.

18. Individual Contribution Summary

Team Member	Primary Responsibilities
Manisha Pradhan	Dataset creation, followup_records.csv, Week 1 architecture planning, final report writing
Sunit Mukherjee	Patient Summary Agent implementation, RAG pipeline (rag_pipeline.py), testing
Reffel Murzello	LangGraph workflow (graph.py), human approval logic, project lead, AWS deployment oversight
Vivek Sharma	Reminder Agent, Streamlit frontend (app.py), appointments.csv, UI/UX design
Pradeep Kumar M	Escalation Agent, error_handler.py, memory.py, integration testing
Mohammed Rohith	AWS EC2/S3 deployment, synthetic documents, config.py, requirements.txt

All team members contributed to code review, testing, documentation review, and the final presentation preparation.